
Probabilistic Numerical Lessons from Integration

Equipped with our survey of both current BQ approaches and their connections to classic quadrature algorithms, we will now tease out some of the intuitions that will inform the remainder of this book.

► 12.1 Why Be Probabilistic?

Discovering the sometimes close connections between probabilistic inference and classical quadrature methods suggests that the probabilistic approach is, at least, *no worse* than the latter, widely accepted, one. Explicitly, probabilistic numerical algorithms can be just as computationally lightweight, just as performant, and just as reliable, as classical methods – because the classical methods *are* implicitly probabilistic.

Having identified these close connections between probabilistic and classical quadrature methods, we will now emphasise their differences: in particular, the unique offerings of the probabilistic approach.

▷ 12.1.1 Aiding Model Customisation

Perhaps the strongest argument for the probabilistic approach is that it exposes the choice of model, as GP mean and covariance, to the user. We have seen how getting this choice right can yield methods that are substantively both more efficient and more trustworthy. If the task is to come up with an efficient quadrature rule for one specific (class of) univariate integrand, the most productive path forward is likely to be choosing a mean function m and covariance k such that samples from the

associated GP match the integrand as closely as possible.¹ Of course, building a strong model entails its own design and running costs (e.g. in updating the GP with new data). For a cheap integrand, like the running example in this chapter, it is likely to be easier to simply use a standard quadrature library. That said, as we discussed for WSABI in §10.2.1, the costs of a strong model should be more properly viewed as an *investment* of computation, that, in many cases, will yield a computational return. That is, the computational overhead is more than compensated by savings in computation due to the increased efficiency of the more accurate model. Such a case is found in applications where each individual evaluation of the integrand has high evaluation cost – perhaps because it involves parsing a large data set, running a costly simulation, or even performing a physical experiment.² Here designing a good prior is likely to be worth the work.

¹ Analysis through the lens of the RKHS adds further analytical tools. For practical purposes, however, samples from the associated Gaussian process are arguably more explicit, and more interpretable.

² A recent example is in Fröhlich et al. (2021).

► 12.2 Monte Carlo as Probabilistic Inference

We close this chapter by returning to Monte Carlo integration for another, closer look. Now that we have realised that several classical quadrature methods can be constructed in a probabilistic fashion, it is natural to ask whether such an interpretation also exists for Monte Carlo integration. The answer is yes, as we will see in this section.

We already noted in passing that, while the trapezoidal rule amounts to the implicit assumption of a continuous integrand, Monte Carlo integration is applicable to any Borel-integrable function. This suggests that a probabilistic formulation of Monte Carlo integration will involve a very vague prior on the integrand. Thus, consider the assumption that the integrand is “white noise” with an unknown mean:

$$f(x) \sim \mathcal{GP}(m, k(x, x')),$$

where $m \in \mathbb{R}$ is an unknown constant, and $k(x, x') = \theta^2 \mathbb{I}(x = x')$ with the indicator function $\mathbb{I}(a) = 1$ if a , 0 elsewhere,³ encoding the assumption that all function values are independent of each other, individually varying around m with variance θ^2 . Following an old idea,⁴ we place the Gaussian prior $p(m) = \mathcal{N}(m; 0, c^{-1})$ on the unknown constant (the precision c will be taken to the limit $c \rightarrow 0$ below). Marginalising over m , we get the prior

$$p(f | c) = \int p(f | m) p(m | c) dm = \mathcal{GP}(f; 0, k(x, x') + c^{-1})$$

³ Readers who are justly worried that this process is not strictly well-defined can consider it as the limit case of the Gaussian process with covariance function

$$k(x, x') = \lim_{\lambda \rightarrow 0} e^{-\frac{(x-x')^2}{2\lambda^2}} \mathbb{I}_{|x-x'| < 3\lambda}.$$

The restriction to a finite support is necessary for the argument below to work, it has no practically relevant effect on this probability measure. The argument here can then be performed for finite values of λ , and the result below obtained as a limit case.

⁴ Blight and Ott (1975); O’Hagan and Kingman (1978).

for the integrand. Under this prior, after evaluations Y at locations $X = [x_1, \dots, x_N]$ (assumed to be pairwise different), the posterior mean and covariance functions on f can be rearranged⁵ to

$$\begin{aligned}\mathbb{E}_{|X,Y}(f(x)) &= R_x^\top \bar{m} + k_{xX} k_{XX}^{-1} Y = R_x^\top \bar{m} + \sum_{i=1}^N \mathbb{I}(x = x_i) y_i, \\ \text{cov}_{|X,Y}(f(x), f(x')) &= k_{xx'} - k_{xX} k_{XX}^{-1} k_{Xx'} + R_x^\top (c + \mathbf{1}^\top k_{XX}^{-1} \mathbf{1})^{-1} R_{x'},\end{aligned}$$

with

$$\begin{aligned}R_x &:= 1 - \mathbf{1}^\top k_{XX}^{-1} k_{xX} = 1 - \sum_{i=1}^N \mathbb{I}(x = x_i), \text{ and} \\ \bar{m} &:= (c + \mathbf{1}^\top k_{XX}^{-1} \mathbf{1})^{-1} \mathbf{1}^\top k_{XX}^{-1} Y = \frac{\theta^{-2}}{c + \theta^{-2}N} \sum_{i=1}^N y_i.\end{aligned}$$

⁵ Rasmussen and Williams (2006), §2.7

The corresponding Gaussian posterior over the *integral* $F = \int_a^b f(x) dx$ has mean and variance

$$\begin{aligned}\mathbb{E}_{|X,Y}(F) &= \int_a^b \mathbb{E}_{|X,Y}(f(x)) dx = (b-a)\bar{m}, \text{ and} \\ \text{var}_{|X,Y}(F) &= \int_a^b \int_a^b \text{cov}_{|X,Y}(f(x), f(x')) dx dx' = \frac{(b-a)^2}{c + \theta^{-2}N}.\end{aligned}$$

Evidently, for the improper limit of $c \rightarrow 0$ (“total ignorance” about the value of m), we get

$$\mathbb{E}_{|X,Y}(F) = \frac{b-a}{N} \sum_{i=1}^N y_i \quad \text{and} \quad \text{var}_{|X,Y}(F) = \frac{\theta^2(b-a)^2}{N}.$$

(12.1)

The variance θ^2 of the “white noise” around the mean m could be inferred using a Gamma prior $p(\theta) = \mathcal{G}(\theta^{-2}; \alpha_0, \beta_0)$, analogously to the procedure described in §11.1.2. For large values of N , and for the improper limit of $\alpha_0 \rightarrow 0, \beta_0 \rightarrow 0$, the resulting Gamma posterior on θ^{-2} would concentrate on its expected value⁶

$$\mathbb{E}_{\mathcal{G}(\theta^{-2}; \alpha_N, \beta_N)}(\theta^{-2}) = \frac{\alpha_N}{\beta_N} = \frac{N}{\sum_i f(x_i)^2},$$

⁶ The *mode* of the Gamma distribution is at $\frac{\alpha-1}{\beta}$, which is often used as an unbiased alternative. For large N , the two estimates are asymptotically equal.

so this probabilistic method’s estimate for the square error of the Monte Carlo estimate matches one that can also be constructed from Eq. (9.5) by other means. We have just found the following theorem.

Theorem 12.1. *The Monte Carlo estimate is the limit as $c \rightarrow 0$ of the maximum a posteriori estimate under the prior*

$$p(f) = \mathcal{GP}(0, \theta^2 \mathbb{I}(x = x') + c^{-1}), \quad (12.2)$$

with arbitrary $\theta \in \mathbb{R}_+$. The corresponding posterior variance estimate on the integral, $\theta^2(b-a)^2/N$, matches the convergence rate of the Monte Carlo estimator. Under this prior, any arbitrary (but non-overlapping) design $X = [x_1, \dots, x_N]$ yields the same posterior variance on F .

A defender of Monte Carlo might argue that its most truly desirable characteristic is the fact that its convergence (see Lemma 9.2) does not depend on the dimension of the problem. Performing well even in high dimension is a laudable goal. However, the statement “if you want your convergence rate to be independent of problem dimension, do your integration with Monte Carlo” is much like the statement “If you want your nail-hammering to be independent of wall hardness, do your hammering with a banana.” We should be sceptical of claims that an approach performs equally well regardless of problem difficulty. An explanation could be that the measure of difficulty is incorrect: perhaps dimensionality is not an accurate means of assessing the challenge of an integral. However, we contend that another possibility is more likely: rather than being equally good for any number of dimensions, Monte Carlo is perhaps better thought of as being equally bad.

Recall from §10.1.2 that the curse of dimensionality results from the increased importance of the model relative to the evaluations. Theorem 12.1 makes it clear that Monte Carlo’s property of dimensionality-independence is achieved by assuming the weakest possible model. With these minimalist modelling assumptions, very little information is gleaned from any given evaluation, requiring Monte Carlo to take a staggering number of evaluations to give good estimates of an integral. As a contrast, Bayesian quadrature opens the door to stronger models for integrands. The strength of a model – its inductive bias – can indeed be a deficiency if it is ill-matched to a particular integrand. However, if the model is well-chosen, it offers great gains in performance. The challenge of high dimension is in finding models suitable for the associated problems. Thus far, Probabilistic Numerics has shone light on this problem of choosing models, and has presented some tools to aid solving it. *It is now up to all of us to do the rest.* Of course, we must acknowledge that contemporary quadrature methods (both probabilistic and classical) do not work well in high-dimensional problems: indeed, they perform far worse than Monte Carlo. However, arguments like those in this chapter show that there is a lot of potential for far better integration algorithms. Such methods can work

only in moving away from Monte Carlo, and its fundamentally limited convergence rate.

For further evidence to this point, we note that even the most general model underlying Monte Carlo integration can actually converge faster if the nodes are not placed at random. Equation (12.1) is independent of the node placement X . So if it is used for guidance of the grid design as in §11.1.3, then any arbitrary node placement yields the same error estimate (as long as no evaluation location is exactly repeated). Since the covariance k assumes that function values are entirely unrelated to each other, a function value at one location carries no information about its neighbourhood, so there is no reason to keep the function values separate from each other.

The tempting conclusion one may draw from Theorem 12.1 is that, because, under this rule, any design rule is equally good, one should just use a *random* set of evaluation nodes. This argument is correct *if* the true integrand is indeed a sample from the extremely irregular prior of Eq. (12.2). But imagine for a moment that, against our prior assumptions, the integrand f happens to be continuous after all. Now consider the choice of a regular grid,

$$X = [a, a + h, a + 2h, \dots, b - h] \quad \text{with} \quad h := \frac{b - a}{N}.$$

Then, the mean estimate from Eq. (12.1) is the Riemann sum

$$\mathbb{E}_{|X,Y}(F) = h \sum_i f(x_i).$$

For functions that are even Lipschitz continuous, this sum converges to the true integral F at a *linear* rate,⁷ $\mathcal{O}(N^{-1})$. That is, the poor performance of Monte Carlo is due not just to its weak model, but its use of random numbers. This insight into the advantage of regular over random node placement is at the heart of *quasi Monte Carlo* methods.⁸ As we have seen above, however, it is possible to attain significantly faster convergence rates by combining non-random evaluation placements with explicit assumptions about the integrand.

⁷ Davis and Rabinowitz (1984), §2.1.6

⁸ E.g. Lemieux (2009)

Thus, from a probabilistic perspective, there is an argument for the integration rule (12.1) if there is no known regularity in the integrand (note that this is virtually never the case in practice, and other integration rules, using more informative priors, should then be preferred). But, even though this prior gives no reason itself to prefer any design X over another one,

it is *still* a good idea to use a regular evaluation rule, on the off chance that the true integrand is continuous after all. From these observations, we draw the contentious conclusion that it is generally a bad idea to introduce random numbers into an otherwise deterministic computation.

► 12.3 A Meditation on Randomness

We offer a few more, provocative, thoughts on the utility of random numbers in the computation of deterministic quantities, and be it only to counter the frequent reflex to equate *probabilistic* methods with *stochastic* ones. We begin with generic arguments against the use of a (pseudo-) random number generator in computation. A PRNG:

1. does not have a coherent decision-theoretic motivation;
2. is worse at its goal of efficient exploration than a probabilistic numerical approach;
3. can be expected to be more computationally expensive in solving a numerical task; and
4. muddles subjectivity and bias.

A decision-theoretic view of randomness What would be the probabilistic numerical view of a PRNG? In such an interpretation, the output must be the minimiser of an expected loss function. However, a PRNG is built with the intent of expressing no preference over its outputs. If we assume that the PRNG is the right approach, and try to determine the associated expected loss, the PRNG's assertion that any possible output is equally good implies that the expected loss surface is uniformly flat. Put another way, this is the assertion that the calculation demanding the PRNG is completely insensitive to its output. This is a strong assumption. An example is found in the extreme underpinnings (see Theorem 12.1) of a Monte Carlo estimate of an integral. In most real applications, there are symmetry-breaking forces at play that render some outputs (at least slightly) better than others. The best such output should always be picked. It is more common that we have a given expected loss function, with unique minimum, over a real interval. If a PRNG is used to select an action in this setting, the action will be sub-optimal, with probability one. If you accept that a random number is a decision, it is necessarily a poor one.

Exploration But what is the right loss function for the task addressed by a PRNG? It is hard to defend a single, one-off, choice being made by a PRNG: that is, to defend the expected loss for such a choice being uniformly flat. A PRNG is perhaps more productively considered as a heuristic for making a *sequence* of decisions. The goal of this sequence (or design), $X = \{x_1, \dots\}$, is to achieve *exploration*, which we will roughly define as providing information about the salient characteristics of some function $f(x)$. As a motivating example, consider $f(x)$ as the integrand of a quadrature problem. A PRNG provides exploration but, remarkably, requires neither knowledge or evaluations of f , nor more than minimal storage of previous choices x . These self-imposed constraints are extreme. First, in many settings, including in the quadrature case considered in this chapter, we have strong priors for f . Second, many problems (again, as in quadrature), render evaluations $f(x)$ pertinent to future choice of x : for instance, a range of x values for which $f(x)$ is observed to be flat is unlikely to require dense sampling. Third, as computational hardware has improved, memory has become increasingly cheap. Is it still reasonable to labour under computational constraints conceived in the 1940s?

The extremity of the PRNG approach is further revealed by broader consideration of the problem it aims to solve. Exploration is arguably necessary for intelligence. For instance, all branches of human creative work involve some degree of exploration. Human exploration, at its best, entails theorising, probing and mapping. This fundamental part of our intelligence is addressed by a consequentially broad and deep toolkit. Random and pseudo-random algorithms, in contrast, are painfully dumb, and are so by design.

To better achieve exploration, the Probabilistic Numerics approach is to explicitly construct a model of what you aim to explore – $f(x)$. This model will serve as a guide to optimally explorative points, avoiding the potential redundancy of randomly sampled points.

Figure 12.1, in contrast to Figure 9.3, is a cartoon indictment of the over-simplicity of a randomised approach.

▷ 12.3.1 Computation Investment

A defender of a PRNG might argue that, in order to define an expected loss, a model and loss function must be defined and maintained. The model, and working with an expected loss, will, of course, require computation and memory: surely this will



Figure 12.1: As argued in Figure 9.3, a numerical algorithm, and its model of the problem, must be simpler and smaller than the problem itself. However, a random numerical algorithm takes this to an undue extreme: its solution is too simple to be effective.

mean that the probabilistic numerical approach will be unable to provide a practical replacement for PRNG⁹. This defender of the PRNG might protest that it is most truly decision-theoretic to choose the random algorithm – that which is actually practical.

The argument here depends on an assumption: that a PRNG will lead to lower memory and computation costs than those of an explicitly probabilistic numerical algorithm. This assumption is false. As above, we would like to reframe the computation used by a probabilistic numerical algorithm not as a burden, but as an investment. That is, the computational expense incurred by a numerical algorithm must be judged at the level of the algorithm as a whole, not at the level of a single iteration. While a probabilistic numerical integrator may spend more computation than a random alternative in selecting a single node, this investment of computation may allow it to converge more quickly to a good estimate of the integral. As such, the probabilistic numerical algorithm, even if its iterations are more computationally expensive, is designed to *reduce* overall computation consumption. In this spirit, we have seen practical examples of the computational reductions possible with Probabilistic Numerics in adaptive Bayesian quadrature (§10.2).

⁹ On a pedantic note, PRNGs themselves have their own – low, but non-zero – overheads. The popular Mersenne twister (Matsumoto and Nishimura, 1998) PRNG, requires 2.5 kilobytes of state and a moderate-length sequence of computational operations to generate its output.

Subjectivity and bias Adversarial settings are often used to motivate the use of PRNGs. More concretely, if your adversary knows which specific piece of computer code (with termination rules and internal tolerances) you intend to use as a quadrature routine, that adversary can present a specific Riemann-integrable integrand that will make your code return the wrong answer. In contrast, the argument in §9.2 showed that the Monte Carlo estimator, placing its evaluation nodes randomly, does not suffer from the same problem: its estimator is guaranteed to converge.

It is unbiased on every Lebesgue-integrable function, and consistent on every square-integrable function.

Let us examine this statement more closely. At its core is the primary property of random numbers: they are *unpredictable*. This property is likely to seem intuitively clear to most readers, but is quite difficult to define precisely. Formal definitions of this property are challenging because they inevitably require a definition of all things *predictable* by a computer, a notion closely connected to computational complexity itself.¹⁰ Thankfully, we do not actually need to understand precisely what is and is not predictable. The point of the Monte Carlo argument is that, *if* we have access to a stream of unpredictable numbers and use it to build an integration method, then no one can design an integrand that will foil our algorithm, simply because that adversary cannot predict where our method will evaluate the integrand.

The possible existence of adversarial problems motivates the construction of *unbiased* Monte Carlo estimators. Unfortunately, ‘bias’ is an overloaded and contested term. In the context of Monte Carlo, ‘unbiased’ simply means that the expected value of a random number is equal to the quantity to be estimated (Lemma 9.2). But this purely technical property draws some emotional power from the (completely unrelated!) association, in common language, of ‘bias’ with unfairness.¹¹ The technical, statistical, definition of bias, that used in defining unbiased estimators, is one term within a particular decomposition of error in predicting a data set. As argued by Jaynes,¹² this term has no fundamental significance to inference. Our goal should simply be to reduce error in the whole. (‘Inductive bias’,¹³ meaning the set of assumptions represented within a machine learning algorithm, represents yet another distinct use of the term. In this sense, Probabilistic Numerics unashamedly *encourages* bias, through the incorporation of useful priors.)

It is important to keep in mind that the users of numerical methods are not adversaries of the methods’ designers. In fact, the relationship is exactly the opposite of adversarial: users often change their problems to be better-suited to numerical methods (as, in deep learning, network architectures are chosen to suit optimisers). As a result, the majority of integrands, and optimisation objective functions, are quite regular functions. Moreover, this regularity is well-characterised and knowable by the numerical algorithm. There may be good use cases for random numbers in areas like cryptography, where a lack of information, unpredictability, is very much the point. But numerical

¹⁰ Church (1940); Kolmogorov (1968); Loveland (1966).

¹¹ Pasquale (2015); O’Neil (2016).

¹² Jaynes and Bretthorst (2003), §17.2

¹³ Mitchell (1980)

tasks are communicated to a numerical algorithm through the error-free, formally complete, channel of source code. In this sense, a numerical algorithm knows as much about its task as is possible. Acknowledging known regularity in a numerical task can lead to substantive performance improvements.

Formal statements of the properties of stochastic algorithms often depend on the use of “real” random numbers. The argument for random numbers in computation finally comes apart when one considers that virtually every instance of Monte Carlo methods does not actually use “real” random numbers, but only pseudo-random numbers. So the argument rests entirely on the user *not* knowing the random number generator used in the computation (and its seed). If anything, this clearly shows that computation is fundamentally conditional on prior knowledge.

We now appeal to aesthetic arguments. It seems odd that the good behaviour of, for example, your optimisation algorithm should require the hiding of information. Equally, it seems unintuitive to artificially inject additional uncertainty into a calculation that is, ultimately, designed to increase certainty.

Here are some sequences of numbers. Can you tell which one is random?

1. 622444111111114444443333333
2. 169399375105820974944592307816
3. 712904263472610590208336044895
4. 100011111101111111100101000001
5. 01110000011100100110111101100011

Here is the solution.

1. This sequence was generated by throwing a six-sided dice seven times and copying down the numbers in sequence of their occurrence, repeating the result of the i th throw i times. This sequence is “random” because it is unpredictable, but it does not pass standard tests of randomness, and a Monte Carlo integration rule using this sequence does not converge in $\mathcal{O}(N^{-1/2})$.
2. These are the 41st to 70th digits of the irrational number π . This sequence, if continued, is devoid of any structure. It is perfectly good for use in a Monte Carlo method. Unless, of course, you tell your reviewer where you got it from. Arising from one of the most widely studied numbers of mathematics, it is also anything but “random”.

3. This sequence was generated by the von Neumann method,¹⁴ a pseudo-random number generator, using the seed 908344. It is the kind of sequence used in real Monte Carlo algorithms, and – now that you know the seed – entirely deterministic. It would have been ok to use this sequence for Monte Carlo estimation up until three sentences ago, when we ripped down the veil of randomness.
4. These are digits taken from a CD-ROM published by George Marsaglia containing random numbers generated using a variety of physical random number generators. These digits (perhaps) were once “random”. But, now that you know where we got them, they are obviously deterministic. But wait! Marsaglia’s CD is actually not available online anymore. Most original copies have long been lost as retiring academics cleared out their offices. We will admit that we do not own a physical copy either. Does this revelation make those digits above random again?
5. To generate this sequence we began with a certain, secret string represented in ASCII, then dropped a coin 32 times to decide whether to flip a bit. We always started with the coin facing heads up. If it landed tails up, we flipped the bit. Here is the catch: we will not tell the height from which the coin was dropped. If we told you the drop was only five millimetres, not enough for the coin to actually turn around, then the sequence is not random. If we told you the height was two meters, then you might conclude the sequence is random. There are an awful lot of “we’s” and “you’s” in this argument. Evidently, randomness is very much a subjective property.

Our arguments here are not purely philosophical. A number of studies¹⁵ have highlighted that the choice of random seed is empirically significant to the performance of popular reinforcement learning algorithms. In scientific fields, discovering that an unexpected variable affects outcomes should lead to work to uncover the mechanism. When the variable is a random seed, however, this approach is impossible: the sole purpose of a pseudo-random generator is to render the outcome an *unpredictable* function of the seed. If we are to be truly scientific, there seems no solution other than to abandon the pseudo-random generator.

Distinct to the seed’s role within the exploration routine, the seed may also dictate the progression of a simulation of an environment. Therein, it may be a way of acknowledging

¹⁴ Von Neumann (1951)

¹⁵ Henderson et al. (2018); Islam et al. (2017); Colas, Sigaud, and Oudeyer (2018); Mania, Guy, and Recht (2018).

environmental randomness. Environmental randomness may similarly limit our ability to improve an agent, but is due to forces outside our control. Even here, randomness is likely to be more epistemic than aleatoric, and science is possible. Indeed, simulations could be chosen to be informative for the training of the agent, rather than naïvely randomly generated.¹⁶

Islam et al. (2017) notes that some reinforcement learning algorithms search for the best random seed, and various authors¹⁷ advocate instead for averaging over a large number of possible seeds. Averaging, in effect, treats the seed as a random variable, an uncontrollable and unknown property of the problem that must be marginalised. However, the seed is not a feature of the problem at all. Importantly, a random seed's existence is owed to the PRNG driving the agents' exploration. In this respect, a seed is a design choice. We typically make such choices to improve the performance of our agents. When we choose to use a pseudo-random generator, we artificially bound our understanding of how to do so. Put another way, the use of a pseudo-random generator introduces a variable, the seed, that can never be improved. We have rendered our performance, in this respect, unimprovable.

¹⁶ Paul, Osborne, and Whiteson (2019)

¹⁷ Mania, Guy, and Recht (2018); Colas, Sigaud, and Oudeyer (2018).